

Das IODA-Architekturmodell

Entwurf eines neuen Architekturmusters auf der Basis der vier fundamental unterschiedenen Ebenen Integration, Operation, Daten und APIs.

Der erste Teil dieser Artikelserie [1] hat die generelle Entwicklung der Architekturmuster von MVC über das Schichtenmodell bis zur Clean Architecture nachvollzogen und begründet, warum das Bedürfnis nach einem neuen Architekturmodell entstand, das die bisher etablierten Modelle ablösen und deren Nachteile beseitigen soll: Das zentrale Problem der funktionalen Abhängigkeiten ist in ihnen nicht wirklich gelöst. Lediglich Linderung durch die Prinzipien Dependency Inversion (DIP) und Inversion of Control (IoC) wurde gebracht – und das zu einem hohen Preis. Komplexität wurde mit Komplexität bekämpft.

Wie könnte nun ein Architekturmuster aussehen, das die Vorteile der bisherigen beibehält und die Nachteile hinter sich lässt? Diese Vorteile sind:

- Definition von „Standardverantwortlichkeiten“, die es voneinander abzugrenzen gilt, um die Verständlichkeit zu erhöhen und grundsätzliche Testbarkeit herzustellen.
- Bewusstmachung der Wichtigkeit von klar ausgerichteten Abhängigkeiten für Verständlichkeit und Testbarkeit.

Und die Nachteile:

- Schlecht testbare funktionale Abhängigkeiten.
- Undurchsichtigkeit durch Entkopplungsaufwand.

Funktionale Abhängigkeiten als Wurzelproblem

Der zentrale Nachteil der erwähnten populären Architekturmuster ist, dass sie auf funktionalen Abhängigkeiten gründen. Die Notwendigkeit zu funktionalen Abhängigkeiten wurde nie hinterfragt. Und die funktionalen Abhängigkeiten haben sich über die Jahrzehnte auch nicht verändert. Sie existieren in der Implementation einer Clean Architecture genauso wie in einer ersten Implementation der Schichtenarchitektur noch ohne DIP/IoC.

Funktionale Abhängigkeiten lassen sich nicht mit DIP/IoC wegzaubern. Ob sie existieren, ist keine Sache der Compilezeitabhängigkeiten, sondern der Laufzeitabhängigkeiten. Wo immer eine Attrappe in einem Test gebaut werden muss, um Logik gezielt testen zu können, existiert eine funktionale Abhängigkeit.

Damit Sie wirklich verstehen, was ich mit funktionaler Abhängigkeit meine, muss ich kurz den schon oft hier benutzten Begriff „Logik“ definieren. Logik sind die Anweisungen, die Funktionalität tatsächlich herstellen. Konkret sind dies vor allem:

- Transformationsanweisungen,
- Kontrollflussanweisungen,
- Input-/Output-Anweisungen.

Durch Abarbeitung nur dieser Anweisungen bekommt Software ihr Verhalten. Wenn Logik passend zusammengestellt ist, funktioniert Software.

Aber was ist zu Anforderungen passende Logik? Das ist schwierig zu sagen. Das ist die Kunst der Programmierung. Da kann leicht etwas schiefgehen. Deshalb muss Logik sorgfältig getestet werden – natürlich automatisiert. Und um das zu erreichen, muss Logik überhaupt erst einmal testbar sein.

```
internal class Program
{
    public static void Main(string[] args) {
        var stopwords = new string[0];
        if (File.Exists("stopwords.txt"))
            stopwords = File.ReadAllLines("stopwords.txt");

        var text = "";
        if (args.Length > 0)
            text = File.ReadAllText(args[0]);
        else {
            Console.WriteLine("Text eingeben: ");
            text = Console.ReadLine();
            if (string.IsNullOrWhiteSpace(text)) return;
        }

        var candidateWords = text.Split(new[] { ' ', '\t', '\n', '\r' },
            StringSplitOptions.RemoveEmptyEntries);

        var countTotal = 0;
        var countDistinct = 0;
        var stopwordsDirectory = new HashSet<string>(stopwords);
        var distinctWords = new HashSet<string>();
        foreach (var wortkandidat in candidateWords) {
            foreach (var m in Regex.Matches(wortkandidat, @"[\w-]*"))
                if (!string.IsNullOrWhiteSpace(m.ToString())) {
                    var word = m.ToString();

                    if (!stopwordsDirectory.Contains(word)) {
                        countTotal++;

                        if (!distinctWords.Contains(word)) {
                            distinctWords.Add(word);
                            countDistinct++;
                        }
                    }
                }
        }

        Console.WriteLine($"{countTotal} Worte, davon {countDistinct} verschieden");
    }
}
```

Unstrukturierter Code ist ein Flickenteppich aus Verantwortlichkeiten (Bild 1)

```

class BusinessLogic : IBusinessLogic
{
    private readonly IStopwords _dal;

    public BusinessLogic(IStopwords dal) { _dal = dal; }

    public (int countTotal, int countDistinct) Count_words(string text) {
        var stopwordsDirectory = _dal.Retrieve();

        var candidateWords = text.Split(new[] { ' ', '\t', '\n', '\r' },
                                         StringSplitOptions.RemoveEmptyEntries);

        var countTotal = 0;
        var countDistinct = 0;
    }
}

```

Zur Laufzeit ruft eine innere Schale eine äußere auf (Bild 2)

Schauen Sie sich Bild 1 an, die Ausgangssituation im ersten Teil des Artikels. Dort sehen Sie nur Logik, sogar Logik mit ganz unterschiedlicher Verantwortlichkeit. Teile der Logik erfüllen die eine Funktion, zum Beispiel Zerlegung des Textes in Wörter, andere Teile der Logik erfüllen eine andere Funktion, etwa die Bestimmung der verschiedenen Wörter. Jede dieser Verantwortlichkeiten kann fehlerbehaftet sein. Jede verdient eigenständige Tests. Das ist in dem Verantwortlichkeitsmischmasch aber nicht machbar. Die Testbarkeit der Verantwortlichkeiten ist gleich null.

Vergleichen Sie damit Bild 2. In der Clean Architecture sind Verantwortlichkeiten getrennt. Der Anteil an Logik, der sich um die Wortzählung kümmert, ist nun in einer eigenen Funktion freigestellt. Damit kann die Wortzählungslogik separat von der Benutzerschnittstellenlogik oder von der Logik zum Laden des Textes aus einer Datei getestet werden.

Allerdings: Die Logik zur Wortzählung ist noch funktional abhängig. Innerhalb von `Count_words()` wird weitere Logik in `Retrieve()` aufgerufen, deren Funktionalität für die Logik in `Count_words()` relevant ist.

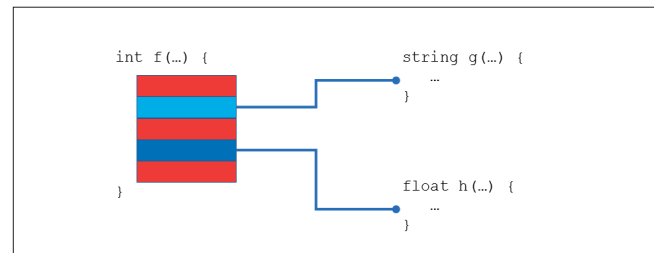
Diese funktionale Abhängigkeit verhegelt die Testbarkeit von `Count_words()`. Durch die Abhängigkeit kann bei einem fehlschlagenden Test nicht eindeutig bestimmt werden, wo das Problem liegt: in der Logik von `Count_words()`, die eigentlich getestet werden soll, oder in der Logik von `Retrieve()`?

Aufgelöst wird diese Unsicherheit durch DIP/IoC: Sie ersetzen die Abhängigkeit im Test einfach durch eine Attrappe.

Erstens macht das aber Aufwand, weil dafür in streng typisierten Sprachen ein Interface nötig ist (DIP). Zweitens macht das Aufwand, weil die Attrappe überhaupt codiert werden muss. Drittens muss die Attrappe verfügbar gemacht werden (IoC). Viertens muss die Attrappe so konfiguriert werden, dass sie zu dem passt, was in der zu testenden Funktion passiert; die Attrappe erzwingt sowohl Wissen über die Verarbeitung in der zu ersetzenden Funktion wie in der zu testenden.

Und schließlich: Funktionale Abhängigkeiten widersprechen ganz prinzipiell sauberer Softwareentwicklung.

- Funktionale Abhängigkeiten verhindern die Einhaltung des Single Responsibility Principle (SRP), weil mit ihnen die abhängige Funktion immer mindestens zwei Verantwortlichkeiten hat: einerseits die Erfüllung einer Funktionalität mit Logik, andererseits den Aufruf von Funktionen in passender Reihenfolge mit den richtigen Parametern. Letzteres nenne ich Integration. Das Resultat: schlechte Testbarkeit, schlechte Verständlichkeit.



In funktionaler Abhängigkeit wechseln sich Logik und Funktionsaufrufe ab (Bild 3)

- Funktionale Abhängigkeiten verhindern die Einhaltung des Prinzips Single Level of Abstraction (SLA). Logik ist per definitionem auf einem niedrigeren Abstraktionsniveau als der Aufruf von Logik mit einer Funktion. Das Resultat: schlechte Verständlichkeit.

Dass man Prinzipien wie DIP/IoC anwenden muss, um die Kompromittierung von SRP und SLA wettzumachen, entschärft die Lage nicht, sondern macht sie schlimmer. Denn DIP/IoC erzeugen künstlich Komplexität, wie selbst Robert C. Martin beklagt [2].

Das zentrale Problem der üblichen Architekturmuster ist mithin Code, der wie in Bild 3 aussieht: In einer Funktion wechseln sich Logik und Aufrufe anderer Funktionen ab.

Dieses Bild stellt ganz deutlich die Laufzeitabhängigkeiten dar. Es ändert sich also auch mit noch so viel DIP/IoC nichts. Durch Anwendung der Prinzipien wird höchstens diese Realität kaschiert – bis Sie in den Tests auf unschöne Weise daran erinnert werden.

Um die Logik von `f()` für sich genommen testen zu können, müssen Sie Attrappen einführen. Und selbst dann wird nur die gesamte Logik getestet, sodass bei einem Fehler unklar ist, ob dieser im ersten, zweiten oder dritten Abschnitt der Logik seine Ursache hat. Die Logikanteile zwischen den funktionalen Abhängigkeiten sind für sich genommen immer noch nicht testbar.

Eine Funktion wie `f()` hat deshalb aus meiner Sicht immer $n+2$ Verantwortlichkeiten. n steht dabei für die Zahl der funktionalen Abhängigkeiten.

- Vor jedem Funktionsaufruf steht Logik (n) und auch nach dem letzten noch mal ($+1$); jede dieser Logik-Passagen tut etwas anderes und stellt eine Verantwortlichkeit dar. Vielleicht ist das nicht immer so, aber diese Verallgemeinerung halte ich für hilfreich. Sie macht das Problem deutlicher als eine Erbsenzählerei, ob wirklich um jeden Funktionsaufruf herum Logik steht.
- Die Integration der ganzen Funktionsaufrufe mit der Logik ist eine separate, übergeordnete Verantwortlichkeit ($+1$).

Die Verantwortlichkeiten, die ich hier meine, sind formal, nicht inhaltlich. Sie ergeben sich allein aus der Struktur. Darüber hinaus kann es also innerhalb der Logikblöcke weitere Verantwortlichkeiten geben.

Die Testbarkeit einer Codebasis steht damit in umgekehrt proportionalem Verhältnis zur funktionalen Abhängigkeit ►

der Funktionen in einer Codebasis. Je mehr Verantwortlichkeiten eine Funktion hat, desto schwerer ist sie testbar. Und sie hat umso mehr Verantwortlichkeiten, je mehr funktionale Abhängigkeiten sie enthält.

Außerdem ist eine solche Codebasis weniger verständlich. Denn funktionale Abhängigkeiten widersprechen nicht nur dem SRP und dem SLA, sondern sorgen auch dafür, dass es beim Funktionsumfang keine Grenze gibt. Solange funktionale Abhängigkeiten okay sind, solange sind auch 100, 1000 oder 10000 Zeilen lange Funktionen okay.

Mit funktionalen Abhängigkeiten wächst der Methodenumfang ganz selbstverständlich bei Bedarf. Wenn zu 100 Zeilen noch 40 dazukommen sollen, dann werden vielleicht 15 Zeilen in eine Funktion extrahiert und aufgerufen – eine funktionale Abhängigkeit ist entstanden – und die 40 neuen Zeilen werden dazugepackt. Am Ende sind in der Funktion 125 Zeilen. Und so geht es weiter und weiter.

Refactorings wie *extract method* scheinen sogar die Hemmschwelle gesenkt zu haben, Funktionen länger und länger zu machen. Es ist ja so einfach, etwas Gutes zu tun, also ein wenig Logik (inklusive funktionaler Abhängigkeiten) zu extrahieren. Da darf man es sich anschließend gönnen, weitere Logik auf allen Ebenen dazuzupacken. Das verringert die Verständlichkeit des Codes weiter, weil dadurch Funktionalität in der Tiefe gestaffelt wird. Jeder Überblick, wie Logik Verhalten erzeugt, geht auf diese Weise verloren. Die Konsequenz können dann nur lange Debugging-Sitzungen sein, in denen die Logik in der Tiefe verfolgt wird.

Wenn Sie es kurz und knackig mögen, lautet mein Vorschlag, Sie berechnen die Testbarkeit einer Funktion so: 1 geteilt durch die Anzahl der Verantwortlichkeiten. Der Wert 1 stellt danach den höchsten Wert für Testbarkeit dar. Eine Funktion hat dann nur eine Verantwortlichkeit. Je weiter der Wert gegen 0 tendiert, desto schlechter ist die Testbarkeit.

Verantwortlichkeiten zu erkennen ist eine Kunst, wenn es um inhaltliche geht. Die Auffassungen können da schon im Detail auseinandergehen. Wie viele Verantwortlichkeiten identifizieren Sie etwa in der hervorgehobenen Logik in **Bild 4**?

```
class BusinessLogicLayer
{
    public static (int countTotal, int countDistinct) Count_words_in_file(string filename) {
        var text = DataAccessLayer.Read_text(filename);
        return Count_words(text);
    }

    public static (int countTotal, int countDistinct) Count_words(string text) {
        var stopwordsDirectory = DataAccessLayer.Load_stopwords();

        var candidateWords = text.Split(new[] { ' ', '\t', '\n', '\r' },
                                       StringSplitOptions.RemoveEmptyEntries);

        var countTotal = 0;
        var countDistinct = 0;

        var distinctWords = new HashSet<string>();
        foreach (var wordCandidate in candidateWords) {
            foreach (var m in Regex.Matches(wordCandidate, @"[\w-]*")) {
                if (!string.IsNullOrEmpty(m.ToString())) {
                    var word = m.ToString();

                    if (!stopwordsDirectory.Contains(word)) {
                        countTotal++;

                        if (!distinctWords.Contains(word)) {
                            distinctWords.Add(word);
                            countDistinct++;
                        }
                    }
                }
            }
        }

        return (countTotal, countDistinct);
    }
}
```

Schlecht testbare Logik trotz Schichtung (**Bild 4**)

Doch das ist aus meiner Sicht schon die Kür. Davor steht die Pflicht, sprich die Erkennung der formalen Verantwortlichkeiten aufgrund funktionaler Abhängigkeit. Nach der obigen Formel sind das in **Bild 4** für *Count_words()* $1+2=3$; die Testbarkeit hat also nur den Wert $1/3=0,33$.

Schauen Sie jetzt zum Vergleich **Bild 5** an. Beide Datenzugriffsmethoden haben keine (!) funktionalen Abhängigkeiten; sie enthalten reine Logik. Ihre formale Verantwortlichkeitszahl ist damit 1 und die Testbarkeit hat den Wert $1/1=1$. Besser geht's nicht. Die Klasse hat keine Abhängigkeiten; sie ist – wie Presenter – ein Blatt im Baum der Laufzeithierarchie. Keine Attrappen sind nötig, um sie zu testen.

Dass der Test von Funktionen, die auf Ressourcen zugreifen, auch nicht ganz einfach ist, steht auf einem anderen Blatt. Für das hiesige Argument ist das nicht wichtig.

Auflösung funktionaler Abhängigkeiten

Wenn funktionale Abhängigkeiten Verständlichkeit und Testbarkeit verringern, wie können sie dann verhindert werden? Soll Software etwa auf Funktionen verzichten?

Natürlich nicht. Funktionen sind wertvolle Strukturierungsmittel für Logik. Um funktionale Abhängigkeiten loszuwerden, brauchen wir davon mehr und nicht etwa weniger.

Dass es jedoch auch ohne funktionale Abhängigkeiten geht, zeigen schon die Funktionen in **Bild 5**. Sie sind Beispiele für eine Art von Funktionen, die ohne auskommt. Ich nenne sie Operationen.

Operationen enthalten ausschließlich Logik. Sie rufen keine weiteren Funktionen auf, über deren Logik sie Kontrolle haben. Dass eine Operation einen Vergleichsoperator benutzt oder einen API-Funktionsaufruf wie *File.ReadAllText()* tätigt, zählt nicht als funktionale Abhängigkeit. Solche Funktionen aufzurufen ist der Job einer Operation. Ob sie das korrekt im Sinne ihrer Verantwortlichkeit tut, gilt es mit automatisierten Tests zu prüfen.

Software kann allerdings nicht nur aus Operationen bestehen. Dann würde die ganze Logik eingepackt in Funktionen nur nebeneinanderstehen, ohne zusammenzuwirken. Operationen müssen also aufgerufen werden. Irgendwo muss es Funktionen geben, die wiederum Operationen als Funktionen aufrufen – allerdings ohne dass dadurch funktionale Abhängigkeiten entstehen. Glücklicherweise ist das ebenfalls möglich. Beispiele dafür sehen Sie in **Bild 6**. *Count_words_in_file()* und *Count_words()* sind Funktionen, die andere Funk-

```
class DataAccess : ITexts, IStopwords
{
    string ITexts.Retrieve(string filename) => File.ReadAllText(filename);

    HashSet<string> IStopwords.Retrieve() {
        const string STOPWORDS_FILENAME = "stopwords.txt";

        var stopwords = new string[0];
        if (File.Exists(STOPWORDS_FILENAME)) {
            stopwords = File.ReadAllLines(STOPWORDS_FILENAME);
            return new HashSet<string>(stopwords);
        }
    }
}
```

Funktionen ohne funktionale Abhängigkeiten sind bestens testbar (**Bild 5**)

```

class UseCase : IUseCase
{
    private readonly IBusinesslogic _bl;
    private readonly ITexts _txt;
    private readonly IPresenter _ui;

    public UseCase(IBusinesslogic bl, ITexts txt, IPresenter ui) {
        _bl = bl;
        _txt = txt;
        _ui = ui;
    }

    public void Count_words_in_file(string filename) {
        var text = _txt.Retrieve(filename);
        Count_words(text);
    }

    public void Count_words(string text) {
        var (countTotal, countDistinct) = _bl.Count_words(text);
        _ui.Display_results(countTotal, countDistinct);
    }
}

```

Der Use Case beim Orchestrieren von Adaptern und Businesslogik (Bild 6)

tionen aufrufen – aber selbst keine Logik enthalten. Auch sie sind damit frei von funktionalen Abhängigkeiten. Diese Art von Funktionen nenne ich Integration.

Integrationen haben formal nur eine Verantwortlichkeit: Sie integrieren, das heißt, sie binden andere Funktionen zu einer größeren zusammen. Aus vielen verschiedenen Funktionen entsteht eine neue auf höherem Abstraktionsniveau.

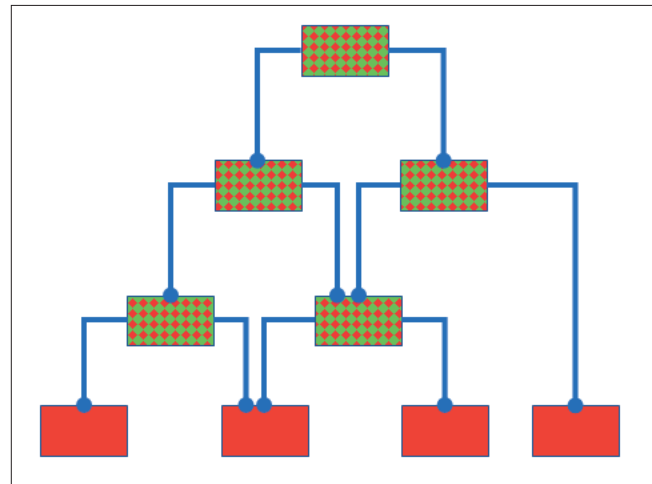
Operationen abstrahieren von Logikdetails, die nötig sind, um Funktionalität herzustellen. Integrationen abstrahieren von den Detailfunktionen, aus denen ein größerer Funktionsbaustein zusammengesetzt ist.

Aber wie steht es mit der Testbarkeit von Integrationen, die mehrere Funktionen aufrufen? Da Integrationen nur eine formale Verantwortlichkeit haben, ist ihre Testbarkeit ebenfalls $1/1=1$. Wenn eine Integration überhaupt automatisiert getestet wird, müssen die aufgerufenen Funktionen nicht durch Attrappen ersetzt werden. Im Gegenteil: Es ist wichtig, dass die realen Funktionen im Test ausgeführt werden, um sicherzustellen, dass die Integration ihren Job tut und ein umfassenderes Verhalten aus Einzelverhalten herstellt.

In vielen Fällen müssen Integrationen aber gar nicht automatisiert getestet werden, weil es keine Logik zu testen gibt und die Sequenz der Funktionsaufrufe offensichtlich korrekt ist im Sinne der Hypothese, dass damit ein bestimmter Effekt erzielt wird. Man kann dann sozusagen einen Induktionsschluss wagen: Wenn die aufgerufenen Funktionen korrekt sind, dann ist die Integration ebenfalls korrekt.

Eine Ausnahme bildet die Integration an der Wurzel einer Aufrufhierarchie. Aber auch dort müssen nicht zwangsläufig Attrappen zum Einsatz kommen.

Funktionen spielen in den traditionellen Architekturmustern keine Rolle. Funktionen sind dort in ihrer Art alle gleich. Auch wird der Begriff Logik nicht benutzt, außer als Namenssuffix. Fast alle Funktionen sind deshalb Hybride mit zweifacher Verantwortlichkeit. Sie operieren und integrieren, sie enthalten Logik und rufen andere Funktionen auf. Eine Ausnahme bilden lediglich die wenigen Funktionen am Fuße der Abhängigkeitshierarchie (Bild 7).



Übliche Architekturmuster erzeugen Hierarchien hybrider Funktionen mit funktionalen Abhängigkeiten (Bild 7)

Und noch mal: Daran ändert der ganze Wirbel um DIP/IoC und die Ausrichtung von Compilezeitabhängigkeiten nichts. Hybride Funktionen sind immer schwer zu testen.

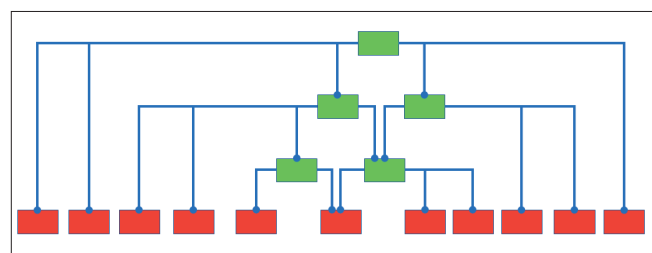
Sobald Funktionen jedoch grundsätzlich in Operationen und Integrationen getrennt werden, entsteht eine andere Hierarchie (Bild 8). In ihr gibt es mehr Funktionen, die jedoch alle kleiner sind als die bisherigen Hybriden. Logik befindet sich darin nur noch in den Blättern.

Mehr Funktionen entstehen, weil die Logikanteile in Hybriden herausgezogen werden müssen in Operationen. Doch das sollte Ihnen keine Angst machen. Im Gegenteil: Endlich ist Logik viel leichter testbar. Endlich sind die Funktionen viel übersichtlicher.

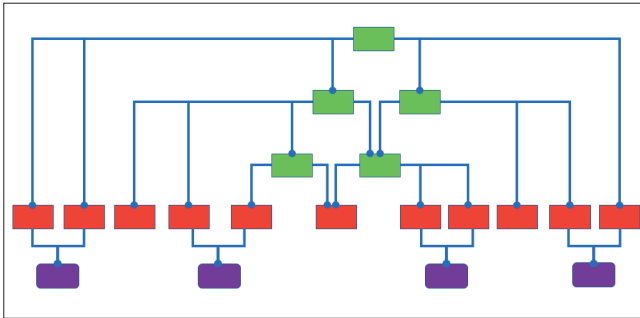
Das Mittel, um eine steigende Zahl von Funktionen in den Griff zu bekommen, liegt außerdem auf der Hand: Klassen. Nicht nur werden also Funktionen kleiner, sondern auch Klassen werden übersichtlicher, weil deren Zahl bei fallendem Umfang steigt.

Durch die Brille der Mainstream-Objektorientierung betrachtet mag das ungewöhnlich aussehen. Doch letztlich ist es das, was auf die eine oder andere Weise immer wieder versucht wurde. Nur sind die bisherigen Empfehlungen sperrig, weil sie auf einem Paradigma basieren, das sie nicht hinterfragen. Es lautet: Funktionen sind Hybride.

Wird das Paradigma jedoch aufgegeben zugunsten einer Unterscheidung von Integrationen und Operationen, tritt ►



Die Grundlage eines neuen Architekturmusters ist eine Hierarchie getrennter Verantwortlichkeitsarten (Bild 8)



Operationen stehen über Daten in Kontakt (Bild 9)

der gewünschte Effekt ganz natürlich ein. Das ist jedenfalls meine Erfahrung nach Jahren der Softwareentwicklung auf diese Weise.

Operationen verbinden

Es gibt noch ein weiteres Problem, unter dem die üblichen Architekturmuster leiden: Sie bieten keinen Platz für Daten. Das ist umso misslicher, als Daten schon früher in Programmablaufplänen und Nassi-Shneiderman-Diagrammen keine Abbildung fanden und somit globalen Daten und daraus resultierend unkontrolliert wachsenden Datenabhängigkeiten Vorschub geleistet wurde.

Auf Architekturebene sind Daten in den Diagrammen entweder gar nicht zu sehen oder sie werden in den Klassen der Schichten und Schalen schlicht automatisch mitgedacht. So wie hybride Funktionen Verantwortlichkeiten vermischen, vermischen die Klassen in den bisherigen Architekturen ebenfalls Verantwortlichkeiten. Sie fassen Funktionen und Daten unterschiedslos zusammen.

Das mag scheinbar durch die Objektorientierung nahegelegt sein, die ja mit dem Anspruch angetreten war, gerade Funktionen und Daten näher zusammenzubringen, um Kapselung zu erreichen. Doch auch wenn dies durch Mittel der Objektorientierung möglich ist, bedeutet das nicht, dass es unsystematisch geschehen sollte. Leider liefern die Architekturmuster hier jedoch keine Hinweise.

Ein modernes Architekturmuster, das über Bisheriges hinausgeht, sollte also das Thema Daten ebenfalls adressieren, um die beschriebene Lücke zu schließen. Nicht nur funktionale Abhängigkeiten sind zu entschärfen beziehungsweise ganz zu eliminieren, sondern auch Datenabhängigkeiten sind zu explizieren.

Zum Glück ist das ganz leicht und ergibt sich in natürlicher Weise aus Bild 8. Integrationen enthalten keine Logik. Wie kommunizieren also die Operationen überhaupt miteinander? Das kann nur über Daten geschehen.

Selbstverständlich ist das auch bei den Hybriden und Operationen in Bild 7 der Fall. Dort fließen Daten in der Hierarchie von oben nach unten und von unten nach oben. Die Logik enthaltenden Funktionen sind hier jedoch alle direkt verbunden. Deshalb sind Daten kein Thema.

Ohne eine Verbindung zwischen Logik in verschiedenen Funktionen stellt sich die Datenfrage allerdings sehr deutlich. Die Antwort darauf zeigt Bild 9.

Operationen sind abhängig von Daten, die sie entweder austauschen (fließende Daten) oder die sie gemeinsam nutzen (stehende Daten, Zustand).

Zwar sind gewöhnlich fast alle Operationen über Daten verbunden, doch interessant sind vor allem Datenstrukturen, die Sie selbst definieren. Solche Datenklassen dürfen natürlich nicht der Forderung widersprechen, auf funktionale Abhängigkeiten zu verzichten. Deshalb sollten Klassen, die Daten sind, keine Methoden besitzen.

Andere Klassen, die Daten haben, publizieren diese nicht. Sie stehen deshalb in Bild 9 nicht auf der untersten Ebene.

In Bezug auf Klassen, die keine Funktionen haben, bestehen keine funktionalen Abhängigkeiten. Das Verhältnis zwischen Integration und Operation setzt sich also fort zu den Daten.

Zumindest zunächst. Denn am Ende ist diese Rigorosität nicht durchzuhalten. Sie würde Möglichkeiten der Objektorientierung zur Kapselung ungenutzt lassen. Deshalb können auch Klassen, die Daten sind, Methoden besitzen – allerdings nur in begrenzter Weise.

Methoden von Datenklassen haben lediglich den Zweck, die Konsistenz der Daten sicherzustellen und den Datenzugriff zu ermöglichen. Stack und Queue sind dafür gute Beispiele: Das sind Klassen, die eine Datenstruktur implementieren (abstrakter Datentyp), aber nur Methoden anbieten, um die „Form“ der Daten aufrechtzuerhalten beziehungsweise zu traversieren.

```
class BusinessLogic {
    public static WordCountResult Count_words(string text, HashSet<string> stopwordsDirectory) {
        var t = Shred(text);
        return new WordCountResult {
            CountTotal = Filter(t.Words, stopwordsDirectory).ToArray().Length,
            CountDistinct = Filter(t.UniqueWords, stopwordsDirectory).ToArray().Length
        };
    }

    private static ShredderedText Shred(string text) {
        var wordCandidates = text.Split(new[] { ' ', '\t', '\n', '\r' }, StringSplitOptions.RemoveEmptyEntries);
        return new ShredderedText(wordCandidates.SelectMany(Normalize));
    }

    IEnumerable<string> Normalize(string wordCandidate) {
        foreach (var m in Regex.Matches(wordCandidate, @"[\w\.-]*"))
            if (!string.IsNullOrEmpty(m.ToString()))
                yield return m.ToString();
    }

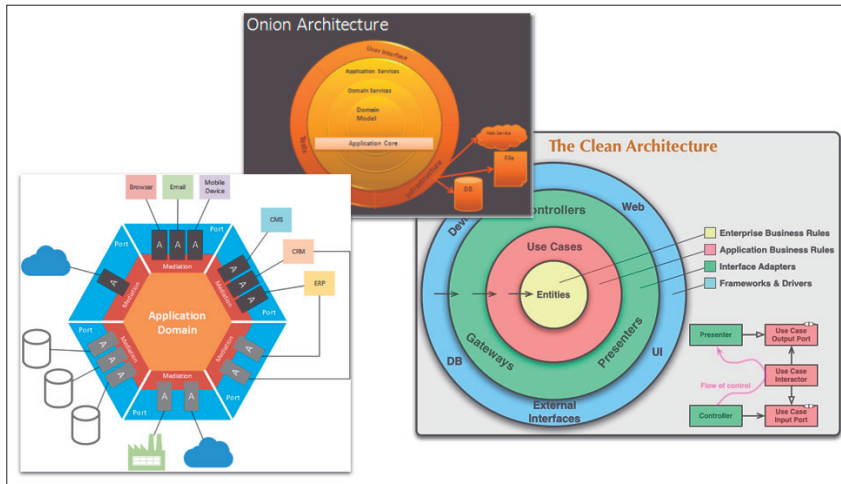
    private static IEnumerable<string> Filter(IEnumerable<string> words, HashSet<string> stopwordsDirectory)
        => words.Where(w => !stopwordsDirectory.Contains(w));
}

class ShredderedText {
    private readonly IEnumerable<string> _words;

    public ShredderedText(IEnumerable<string> words) { _words = words; }

    public string[] Words => _words.ToArray();
    public string[] UniqueWords => _words.Distinct().ToArray();
}
```

Eine Datenklasse verbindet Operationen (Bild 10)



Konzentrische Architekturmuster (Bild 11)

Wenn Operationen Methoden auf solchen Datenobjekten benutzen, dann sehe ich darin keine belastende funktionale Abhängigkeit. Für Datenstrukturen müssen keine Abstraktionen definiert werden, um sie in Tests zu ersetzen. Datenklassen, die in dieser Weise funktional einfach und fokussiert gehalten werden, erzeugen keine zusätzliche Komplexität. Sie machen nichts schwieriger zu verstehen. Im Gegenteil: Immer wieder hilft es, Logik aus Operationen herauszuziehen in Datenstrukturen, um Operationen und Integrationen zu entzerren.

Zur Veranschaulichung habe ich einmal die Businesslogik unseres Beispiels in diesem Sinne verändert. Integrationen und Operationen sind in Bild 10 sauber getrennt. Außerdem ist *ShreddedText* als Datenklasse mit minimaler Logik herausgezogen und dient der Kommunikation zwischen den Operationen *Shred()* und *Filter()*.

Dass die Integration Eigenschaften auf *ShreddedText* aufruft, das Datenobjekt selbst also gar nicht nach *Filter()* hineinfließt, ist lediglich eine naheliegende Optimierung. Sie ändert nichts am Zweck der Datenklasse, Daten zu sein und Operationen zu verbinden.

Verbindung zur Außenwelt

In den konzentrischen Architekturmustern gibt es noch einen Aspekt, der hinzugekommen ist beziehungsweise gegenüber

MVC und dem Schichtenmodell deutlicher gemacht wurde: die Kommunikation mit der Außenwelt.

I/O-Operationen oder allgemeiner API-Aufrufe auf Bibliotheken und Frameworks, die nicht unter Ihrer Kontrolle stehen, sind etwas Besonderes. Einerseits geht von dort immer wieder eine Behinderung von Tests aus, weil I/O vergleichsweise impermanent ist und/oder Einrichtungsaufwand bedeutet. Andererseits unterliegen externe Bibliotheken unkontrollierten Änderungen oder sollen gar austauschbar sein. Beides legt nahe, API-Aufrufe in möglichst wenigen Modulen zu isolieren. Hier kommt das Prinzipienduo DIP/ IoC dann tatsächlich zu gutem Einsatz.

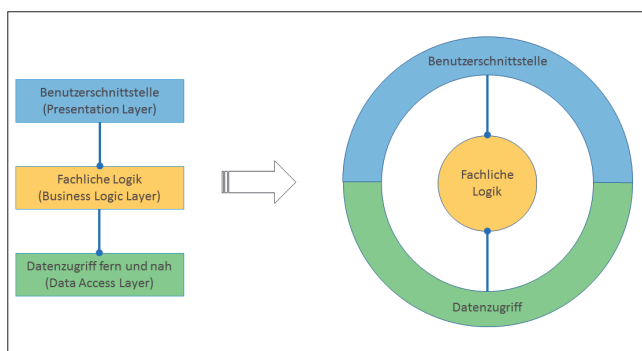
Bei den konzentrischen Architekturmustern ist zumindest I/O deutlich auf die äußere Schale beschränkt, die noch zum Softwaresystem gehört. In Bild 11 finden Sie zu diesem Zweck Adapter in der hexagonalen Architektur und Controller, Gateways, Presenter in der Clean Architecture.

Das Schichtenmodell ist in dieser Hinsicht wenig explizit und undifferenziert. Benutzerschnittstelle und Datenzugriff scheinen grundsätzlich verschieden zu sein (Bild 12). Das macht auch eine seiner Schwächen aus, die die konzentrischen Muster ausgleichen.

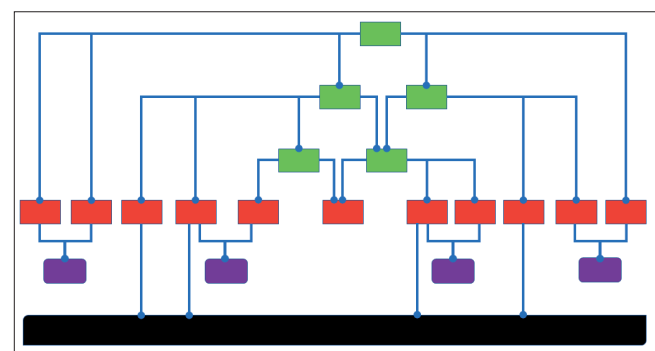
Ein besseres Architekturmodell sollte diese Klarheit und Differenzierung natürlich erhalten. In einer Funktionshierarchie ohne funktionale Abhängigkeiten ist das jedoch sehr leicht. API-Aufrufe gehören zur Logik, und Logik findet sich nur in Operationen (Bild 13).

Wie oben zugestanden, können zwar auch Datenklassen Logik enthalten, doch die ist bewusst inhaltlich auf die Daten in den Strukturen begrenzt und darf formal gerade keinen I/O beziehungsweise keine speziellen API-Aufrufe enthalten. Datenklassen sollen möglichst stabil sein, da von ihnen gewöhnlich viele Funktionen abhängig sind. Logik ist dem abträglich, umso mehr, wenn sie auch noch von APIs abhängig wäre.

Die Klasse *Dataaccess* in Bild 5 liefert zwei Beispiele für Operationen mit I/O-Logik. ►



Konzentrische Abhängigkeiten ergeben mehr Sinn (Bild 12)



Nur Operationen rufen externe APIs auf (Bild 13)

Ein neues Architekturmuster

Nun ist alles beisammen für ein neues Architekturmuster, wie ich es mir vorstelle, um den Ballast der bisherigen abzuwerfen. Ich nenne es die IODA-Architektur, wegen der darin fundamental unterschiedenen Ebenen (Bild 14).

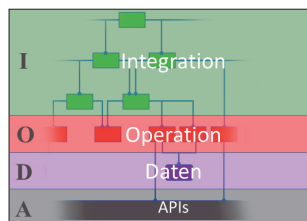
Bitte beachten Sie, dass es in diesem Architekturmuster zunächst keine inhaltlichen Verantwortlichkeiten gibt. Auch darin unterscheidet es sich vom Üblichen. Insofern beansprucht die IODA-Architektur tatsächlich Universalität. Ob Game oder CRM oder Buchhaltung: Jeder Software steht diese „Anatomie“ gut zu Gesicht, glaube ich.

Unterschieden werden zunächst lediglich formale Verantwortlichkeiten. Integration hat einen ganz anderen Auftrag als Operation, und Operation hat einen ganz anderen Auftrag als Daten. Und APIs sind Ihrer Verfügung ganz entzogen; Sie müssen sie nehmen, wie sie sind.

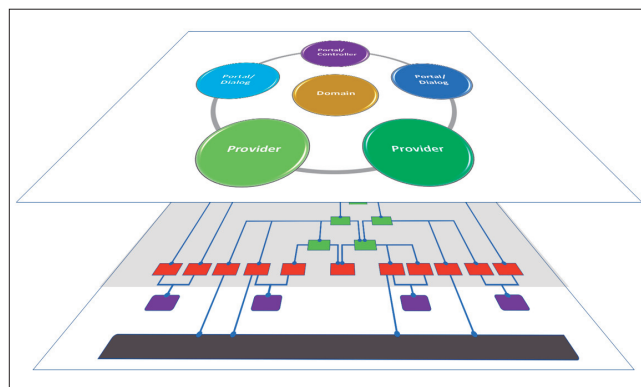
Diese Grundstruktur ist jedoch nur eine mögliche, wenn auch die vordringliche Perspektive auf Software. Sie definiert, wie Code aufgebaut werden sollte, unabhängig davon, was seine lösungsbezogene Aufgabe ist.

In dieser Perspektive geht es zunächst auch nur um Funktionen und Datenstrukturen. Das halte ich aber für wichtig, weil damit deutlich in den Blick genommen ist, was Software ausmacht: Logik, deren Verhalten darin besteht, Daten zu transformieren.

Wenn ein Architekturmuster die zwei Seiten der Softwaremünze – Logik und Daten – in seinem Bild nicht deutlich repräsentiert, dann, glaube ich, fehlt etwas. Die IODA-Architektur tut das. Und sie macht eine klare Aussage darüber, wie Logik in Funktionen zu verpacken ist, um Verständlichkeit und Testbarkeit auf ein Mindestniveau zu heben.



Die Ebenen des neuen Architekturmusters (Bild 14)



Die zwei Ebenen der IODA-Architektur (Bild 16)

Aber die IODA-Architektur lässt die inhaltlichen Verantwortlichkeiten nicht aus dem Blick. Orthogonal zu den Ebenen in Bild 14 gibt es eine ebenfalls konzentrische Darstellung von Software (Bild 15). Hier trifft IODA eine Aussage zum Muster der Verantwortlichkeiten.

Zu unterscheiden ist zunächst das Softwaresystem von der Umwelt. Der Code, den Sie schreiben, gehört zum Softwaresystem. Alles andere ist Bestandteil der Umwelt. Ange deutet wird das durch die Kreislinie, die das Innen vom Außen trennt.

Auf der Kreislinie angesiedelt sind Grundverantwortlichkeiten, die alle zur Kategorie Adapter gehören. Sie kapseln insbesondere I/O-Logik, aber auch andere API-Aufrufe. Adapter verbinden das Innen mit dem Außen.

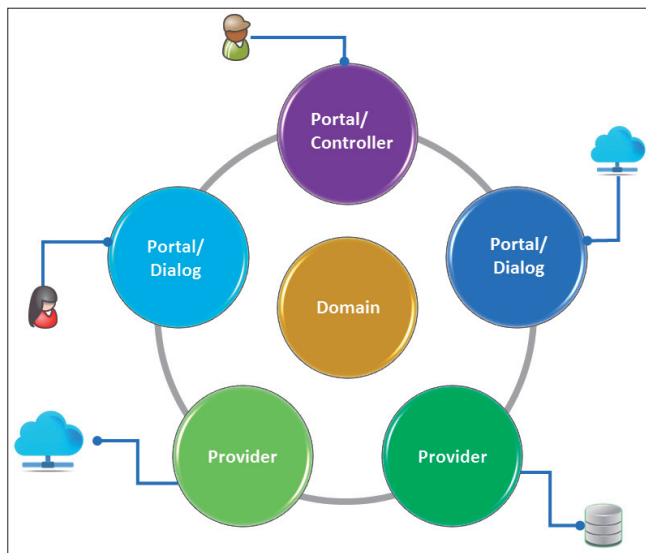
Anders als die hexagonale Architektur teilt die IODA-Architektur die Adapter jedoch in zwei Gruppen: Portale und Provider. Durch Portale wird das Softwaresystem von der Umwelt gesteuert. Durch Provider greift das Softwaresystem auf Umweltressourcen zu, von denen es abhängig ist. Der Kontakt mit der Umwelt ist also asymmetrisch.

Benutzer triggern Logik über Portale, indem sie dort Input-Daten anliefern. Ebenso erhalten Benutzer über Portale Output-Daten zurück. Benutzer können Menschen sein oder andere Softwaresysteme.

Portale gibt es allerdings in zwei Ausprägungen: als Controller und als Dialoge. Controller machen ein Softwaresystem über Entry Points zugänglich. *Main()* ist ein solcher Entry Point, die Klasse *Program* also ein Controller. Mittels Controllern werden Softwaresysteme gestartet oder zumindest aus einem inaktiven Zustand „geweckt“.

Dialoge hingegen bieten weitere Trigger im Rahmen des Aufrufs über einen Entry Point. Denken Sie an einen Button auf einem WPF-Fenster. Das Fenster ist ein Dialog, der mehr oder weniger direkt innerhalb von *Main()* geöffnet wurde. In Dialogen wartet Software aktiv auf einen Input durch einen Benutzer.

Die Unterscheidung zwischen Portalen und Providern ist nützlich, weil damit sprachlich das asymmetrische Verhältnis eines Softwaresystems zu seiner Umwelt dokumentiert wird. Auf der einen Seite wird es von der Umwelt mittels APIs gesteuert, auf der anderen Seite steuert es selbst die Umwelt mittels APIs.



Die inhaltliche Dimension der IODA-Architektur (Bild 15)

Die Unterscheidung zwischen Controllern und Dialogen wiederum ist nützlich, weil damit sprachlich ihre Position in der IODA-Hierarchie dokumentiert wird. Controller als Module haben integrierende Funktion, Dialoge als Module haben operierende Funktion.

Schließlich steht die Domain in der Mitte. Sie betont das, worum es eigentlich geht: die Fachlichkeit. Die darum kreisenden Adapter können sich in Art und Zahl ständig ändern. Doch von der Domain ist anzunehmen, dass sie davon recht wenig betroffen im Kern eines Softwaresystems ruht.

Die Adapter haben sogar die Aufgabe, genau das sicherzustellen. Sie bilden quasi einen Schutzwall gegenüber der Umwelt, zu der auch die zahllosen APIs gehören; sie schaffen auf diese Weise einen Innenraum, der in eigener Geschwindigkeit evolvieren soll.

Aus dieser Perspektive betrachtet, ist Software auch in der IODA-Architektur in klare Verantwortlichkeiten geteilt:

- Adapter
 - Portale
 - Controller
 - Dialoge
 - Provider
- Domain

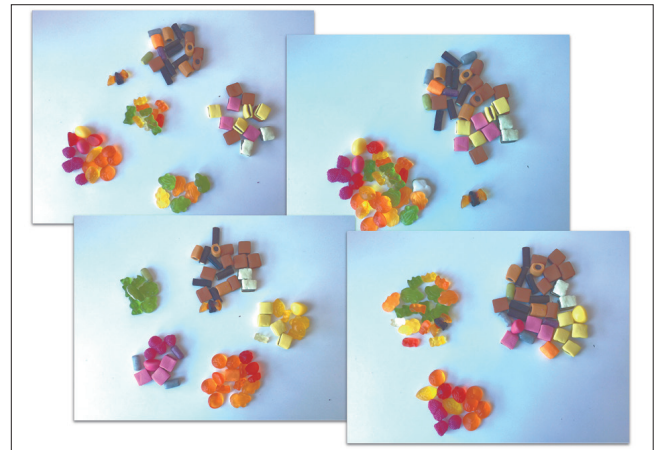
Das ist differenzierter als bei MVC oder beim Schichtenmodell oder auch der hexagonalen Architektur. Doch ich finde es immer noch sehr allgemein und deshalb universell. Ob Twitter-Klon, Game oder Hausverwaltung: Diese Verantwortlichkeiten lassen sich immer finden.

Wie in der Clean Architecture ist auch bei IODA zu erwarten, dass äußere Verantwortlichkeiten in stärkerem Maß Veränderungen unterliegen als innere.

Allerdings – und das ist der zentrale Unterschied zu allen anderen Architekturmodellen – stehen die Verantwortlichkeiten in keiner funktionalen Abhängigkeit!

Es gibt keine Abhängigkeitspfeile, die nur von außen nach innen zeigen dürfen. Ein Dialog braucht keinen Provider, um seinen Job zu machen. Die Domain ist von keinem Provider abhängig.

Verbunden werden die Verantwortlichkeiten vielmehr durch integrierende Beziehungshierarchien – die „zufällig“



Abstraktion durch Kategorisierung (Bild 18)

ihren Ausgang von den Controllern nehmen. „Zufällig“ sage ich, weil das nicht Absicht des Modells ist, sondern leider technische Notwendigkeit. Wenn ein Entry Point eines Controllers getriggert wird, bedeutet das gewöhnlich eine Instanzierung des Controllers. Vorher ist also kein Code gelaufen, der integrierend hätte wirken können.

An dieser Stelle ist das aber ein Detail, um das Sie sich nicht sorgen sollten. Der Grundsatz bleibt: Es gibt keine funktionalen Abhängigkeiten. Die IODA-Architektur hat zwei Ebenen, wobei die Hierarchie die grundlegende ist (Bild 16). Sie ermöglicht, dass in der darüberliegenden Ebene keine funktionalen Abhängigkeiten existieren.

In der IODA-Architektur ist getrennt, was getrennt sein muss: formale und inhaltliche Verantwortlichkeiten. Die überkommenen Architekturmuster hingegen vermischen hier die Verantwortlichkeiten – mit negativem Effekt für Verständlichkeit und Testbarkeit.

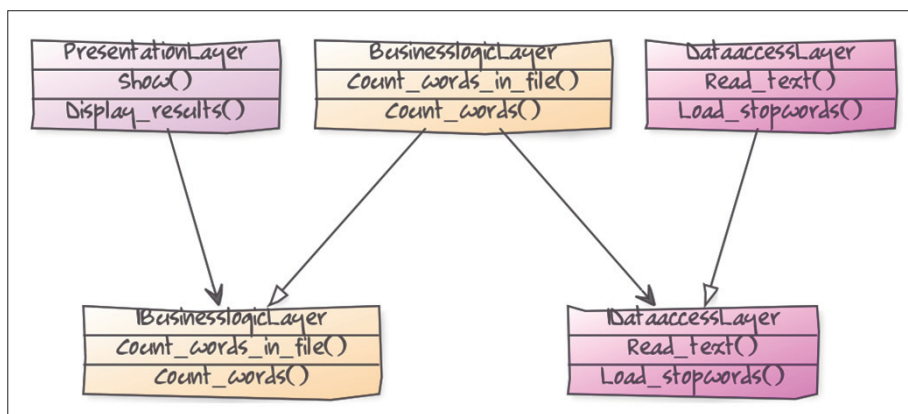
Echt abstrakt

Die Hierarchie der Integration fügt der IODA-Architektur noch einen Aspekt hinzu, der bei den bisherigen Modellen unberücksichtigt geblieben ist: Abstraktion.

Von Abstraktion ist zwar in DIP die Rede – eine Verantwortlichkeit soll nicht direkt von einer anderen abhängig sein, sondern von einer Abstraktion (Bild 17) –, doch diese Art der Abstraktion meine ich nicht.

Verwirrend, oder? Um es zu verstehen, hier meine Sichtweise auf Abstraktion. Es gibt mindestens zwei verschiedene Arten:

- Kategorisierung: Verschiedene Elemente werden auf Gemeinsamkeiten untersucht. Ist eine vorhanden, können die Elemente unter dieser Gemeinsamkeit zusammengefasst werden. Eine Kategorie wird gebildet. Beispiele dafür zeigt Bild 18. Der Inhalt einer Haribo-Color-Rado-Tüte wur- ►



Über Interfaces entkoppelte Schichten (Bild 17)

de nach verschiedenen Kriterien kategorisiert, wie zum Beispiel Geschmack oder Form.

- **Komposition:** Verschiedene Elemente werden zu einem Ganzen vereint, eben weil sie verschieden sind. Im Verein stellen sie auf höherer Ebene allerdings etwas Neues dar. Ein Beispiel dafür zeigt **Bild 19**: Zahnräder, Schrauben, Zeiger und Federn ergeben zusammen ein Uhrwerk.



Abstraktion durch Komposition (Bild 19)

In der Softwareentwicklung finden wir beide Arten der Abstraktion. Der Kategorisierung dienen Klassen und Interfaces. Diese Art der Abstraktion wird bei DIP aufgerufen.

Kategorisierung ist natürlich nützlich. Sie schafft Überblick in der Vielfalt. Aber Kategorisierung ist nicht funktional.

Komposition hingegen ist funktional. Das Neue, das aus der Komposition entsteht, stellt auf höherer Ebene wieder ein nützliches Ganzes dar. Entweder sind die darin eingegangenen Einzelteile alleine nicht nützlich oder es ist umständlich, mit ihnen einen Zweck zu verfolgen. Die Komposition jedoch verbirgt diese Details. Die Komposition vereinfacht, sie macht produktiver.

Beispiel dafür sind die Integrationen in der IODA-Architektur. Sie ziehen Operationen (oder auch andere Integrationen) zu etwas funktional Größerem zusammen. Integrationen bringen etwas auf den Punkt.

Abstraktion durch Komposition findet sich in den bisherigen Architekturmodellen nicht. Es ist ein Irrglaube, dass es bei den Schichten im Schichtenmodell um Abstraktion ginge. Ebenso wenig ist eine Entity im Kern der Clean Architecture abstrakter als ein Use Case oder ein Presenter in den Schalen drumherum.

Eine gewisse Abstraktion durch Kategorisierung enthalten die Architekturmodelle jedoch trotzdem, weil die Blöcke beziehungsweise Schalen mit ihren Verantwortlichkeiten für eine größere Anzahl von Modulen stehen, die demselben Zweck dienen.

Aber wie gesagt: Auch wenn Kategorisierung die Verständlichkeit erhöht, auf die Produktivität hat sie damit relativ geringen Einfluss. Ganz anders die Komposition! Das beweist die Geschichte der Programmiersprachen. Das beweist der Erfolg des OSI-Modells. Das beweist die Entwicklung von Frameworks. Eine Funktion wie `File.ReadAllLines()` ist dafür ein triviales, dennoch passendes Beispiel. Die Produktivität unserer Branche ist dramatisch gestiegen, weil wir Kompositionen zu neuen Kompositionen vereint haben und so auf immer höhere Ebenen der Einfachheit durch Abstraktion gestiegen sind.

Leider hat das in den Softwarearchitekturmustern bisher keinen Eingang gefunden. Meine Vermutung ist deshalb, dass die sinkende Produktivität innerhalb von Projekten zumindest zum Teil darauf zurückzuführen ist. Es wird nicht systematisch nach Kompositionen als Abstraktionen gesucht, um produktiver zu werden. Da wird zwar von Wiederver-

wendbarkeit geredet – doch das Mittel Klasse ist dafür viel zu schwerfällig, und die vorzeitige Generalisierung lauert überall.

In der IODA-Architektur hingegen ist Abstraktion durch Komposition ganz einfach: Wo eine Funktion andere integriert, steht eine neue Funktion zur Verfügung, die produktiver macht, weil sie Details verbirgt (vergleiche dazu meinen Blog-Artikel „Stratified Design over Layered Design“ [3]). Und sie tut das auch noch in einer Weise, die die Testbarkeit nicht kompromittiert.

Wenn eine Funktion eine andere kennt, dann dient das der Abstraktion durch Komposition. Eine integrierende Funktion zieht darüber verschiedene Funktionen zu einem neuen, nützlichen und besser verständlichen Ganzen zusammen.

Es gibt sonst keinen Grund, warum eine Funktion eine andere aufrufen sollte. Da ist IODA ganz strikt. Funktionen mit ganz verschiedenen Aufgaben sollen einander nicht kennen. Das ist aber in den bisherigen Architekturmodellen der Fall. Ob das mit zwischengeschobener kategorisierender Abstraktion in Form eines Interface geschieht oder nicht, spielt dabei keine Rolle.

In einer IODA-Architektur verlaufen Pfeile also vom Abstrakten zum Konkreten, vom Ganzen zu den Teilen, vom hohen zum niedrigen Kompositionsniveau. Das ist grundsätzlich anders als in den bisherigen Architekturmustern.

Bisher verliefen Pfeile vom einen zum anderen in Richtung der Herstellung von Verhalten. Pfeile haben vor allem Verschiedenes auf demselben Kompositionsniveau verbunden. Mit IODA ändert sich das. Dort verlaufen Pfeile im Sinne von Funktionsaufrufen nicht zwischen Verschiedenem, sondern zwischen Gleichem – nur liegt dieses Gleiche auf unterschiedlichen Abstraktionsniveaus. ■

[1] Ralf Westphal, *IODA-Architektur, Teil 1, Eine Kritik bisheriger Architekturmodelle*, dotnetpro 8/2018, Seite 24 ff., www.dotnetpro.de/A1808IODA

[2] Robert C. Martin, *Clean Architecture, A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017, ISBN 978-0-13-449416-6

[3] Ralf Westphal, *Stratified Design over Layered Design*, <http://ccd-school.de/2017/06/stratified-design-over-layered-design>



Ralf Westphal

ist freiberuflicher Berater, Referent, Autor und Trainer für Themen rund um Softwarearchitektur und die Organisation von Softwareteams. Er ist Mitgründer von Clean Code Developer (CCD) und propagiert kontinuierliches Lernen.

info@ralfw.de

dnpCode

A1809IODA